

Programming for Business (CL-2016)

LAB INSTRUCTOR
OWAIS HAMEED

Week 04 Objectives:

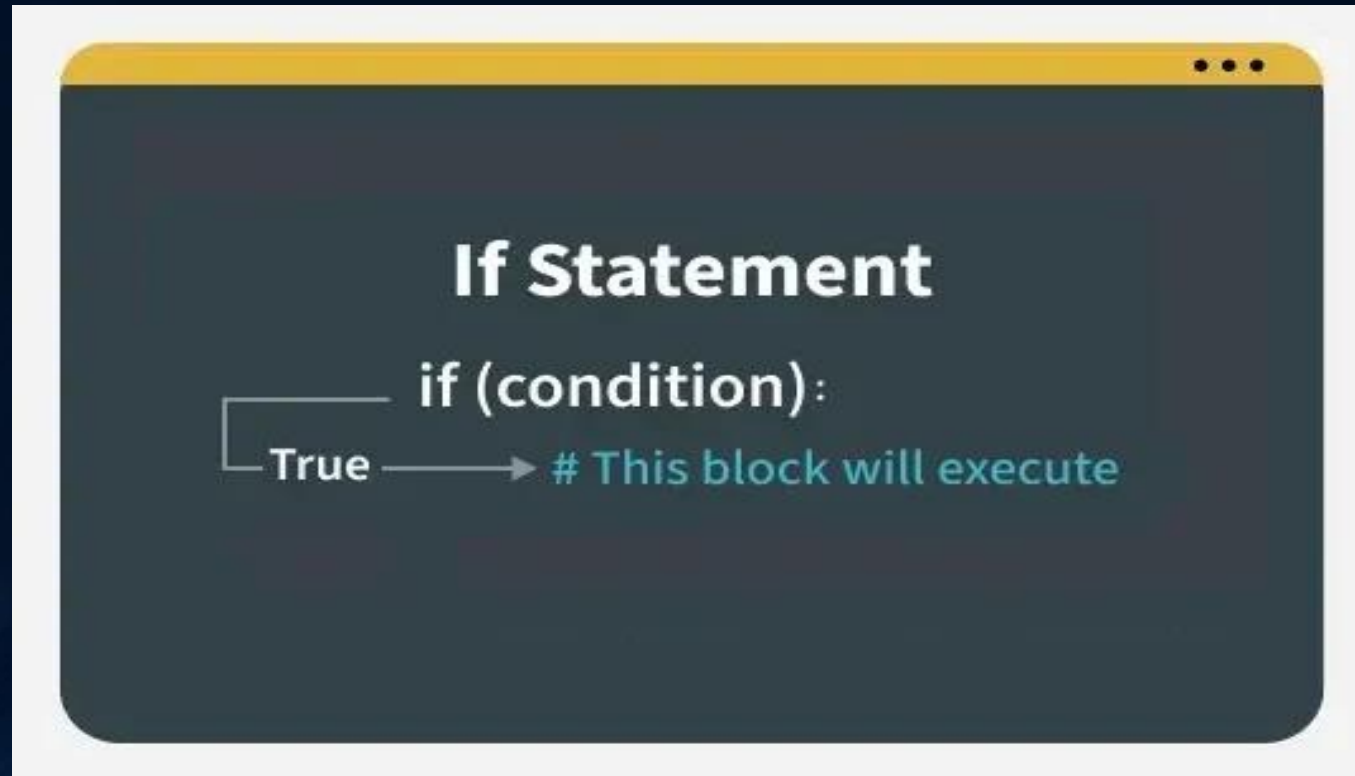
- Conditional Statements
 - If
 - else
 - elif
 - Nested
 - Logical operations
 - Ternary Conditional Statement
 - Match Case

Conditional Statements in Python

- Conditional statements in Python are used to execute certain blocks of code based on specific conditions.
- These statements help control the flow of a program, making it behave differently in different situations.

If Conditional Statement

- If statement is the simplest form of a conditional statement. It executes a block of code if the given condition is true.



Example

- age = 20

```
if age >= 18:
```

```
    print("Eligible to vote.")
```

- number = 15

```
if number > 0:
```

```
    print("The number is positive")
```

Multiple Statements in If Block

- You can have multiple statements inside an if block. All statements must be indented at the same level.

Example

- Multiple statements in an if block:

```
age = 20
if age >= 18:
    print("You are an adult")
    print("You can vote")
    print("You have full legal rights")
```


Using Variables in Conditions

- Boolean variables can be used directly in if statements without comparison operators.
- Example
- Using a boolean variable:
- ```
is_logged_in = True
if is_logged_in:
 print("Welcome back!")
```

# Short Hand if

- Short-hand if statement allows us to write a single-line if statement.

Example

```
age = 19
```

```
if age > 18: print("Eligible to Vote.")
```

- This is a compact way to write an if statement. It executes print statement if the condition is true.



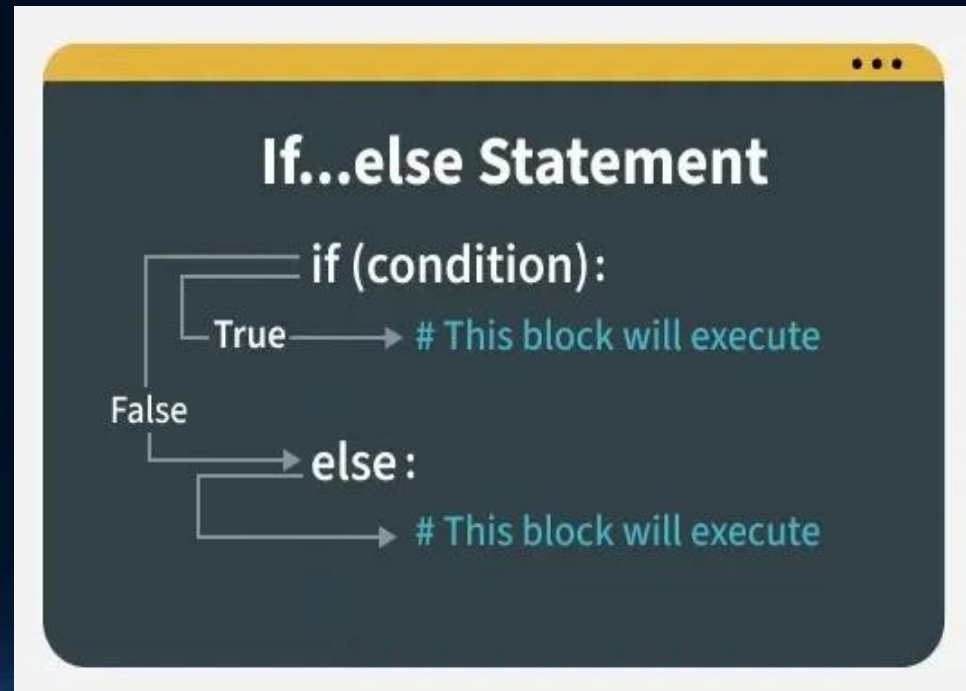
# Indentation

- Python relies on indentation (whitespace at the beginning of a line) to define scope in the code. Other programming languages often use curly-brackets for this purpose.
- Example
- If statement, without indentation (will raise an error):

```
a = 33
b = 200
if b > a:
print("b is greater than a")
```

# If else Conditional Statement

- If Else allows us to specify a block of code that will execute if the condition(s) associated with an if or elif statement evaluates to False.
- Else block provides a way to handle all other cases that don't meet the specified conditions.



# Example

```
age = 10
```

```
if age <= 12:
```

```
 print("Travel for free.")
```

```
else:
```

```
 print("Pay for ticket.")
```

# Example

- `a = 200`
- `b = 33`
- `if b > a:`
- `print("b is greater than a")`
- `else:`
- `print("b is not greater than a")`

# Example

Validating user input:

- `username = "Ali"`
- `if len(username) > 0:`
- `print(f"Welcome, {username}!")`
- `else:`
- `print("Error: Username cannot be empty")`

# Short Hand if-else

- The short-hand if-else statement allows us to write a single-line if-else statement.

```
marks = 45
```

```
res = "Pass" if marks >= 40 else "Fail"
```

```
print(f"Result: {res}")
```



# Multiple Conditions on One Line

- You can chain conditional expressions, but keep it short so it stays readable:

## Example

- One line, three outcomes:

```
a = 330
```

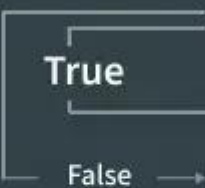
```
b = 330
```

```
print("A") if a > b else print("=") if a == b else print("B")
```

# elif Statement

- elif statement in Python stands for "else if." It allows us to check multiple conditions, providing a way to execute different blocks of code based on which condition is true.
- Using elif statements makes our code more readable and efficient by eliminating the need for multiple nested if statements.

## 01 | if 1st Condition is true



```
graph TD; A["if (condition):"] -- True --> B["# This block will execute"]; A -- False --> C["elif (condition):"]; C --> D["# This block will execute"]; C --> E["else:"]; E --> F["# This block will execute"];
```

if (condition):  
    True → # This block will execute  
    False → elif (condition):  
            # This block will execute  
            else:  
                # This block will execute

## 02 | if 2nd Condition is true

```
if (condition):
 True → # This block will execute
elif (condition):
 True → # This block will execute
else:
 # This block will execute
```

### 03 | if all Condition are false



# Example

- `age = 25`
- `if age <= 12:`
- `print("Child.")`
- `elif age <= 19:`
- `print("Teenager.")`
- `elif age <= 35:`
- `print("Young adult.")`
- `else:`
- `print("Adult.")`



# Testing multiple conditions:

- `score = 75`

```
if score >= 90:
 print("Grade: A")
elif score >= 80:
 print("Grade: B")
elif score >= 70:
 print("Grade: C")
elif score >= 60:
 print("Grade: D")
```



# Example

## Day of the week checker:

- `day = 3`

```
if day == 1:
 print("Monday")
elif day == 2:
 print("Tuesday")
elif day == 3:
 print("Wednesday")
elif day == 4:
 print("Thursday")
elif day == 5:
 print("Friday")
elif day == 6:
 print("Saturday")
elif day == 7:
 print("Sunday")
```

# Example

## Temperature classifier:

- temperature = 22

```
if temperature > 30:
 print("It's hot outside!")
elif temperature > 20:
 print("It's warm outside")
elif temperature > 10:
 print("It's cool outside")
else:
 print("It's cold outside!")
```

# Python Logical Operators

## Example

- Test if a is greater than b, AND if c is greater than a:
- a = 200
- b = 33
- c = 500
- if a > b and c > a:
- print("Both conditions are True")

# Example

- Test if a is greater than b, OR if a is greater than c:
- a = 200
- b = 33
- c = 500
- if a > b or a > c:
- `print("At least one of the conditions is True")`

# Example

- Test if a is NOT greater than b:
- a = 33
- b = 200
- if not a > b:
- `print("a is NOT greater than b")`

# Combining Multiple Operators

- You can combine multiple logical operators in a single expression. Python evaluates not first, then and, then or.
- Combining and, or, and not:
- Example

```
age = 25
```

```
is_student = False
```

```
has_discount_code = True
```

```
if (age < 18 or age > 65) and not is_student or has_discount_code:
 print("Discount applies!")
```

# Using Parentheses for Clarity

- When combining multiple logical operators, use parentheses to make your intentions clear and control the order of evaluation.
- Example
- Using parentheses for complex conditions:
- ```
temperature = 25  
is_raining = False  
is_weekend = True  
  
if (temperature > 20 and not is_raining) or is_weekend:  
    print("Great day for outdoor activities!")
```


Example

- User authentication check:
- `username = "fastkhi"`
`password = "secret123"`
`is_verified = True`

```
if username and password and is_verified:  
    print("Login successful")  
else:  
    print("Login failed")
```

Example

- Range checking with logical operators:

```
score = 85
```

```
if score >= 0 and score <= 100:
```

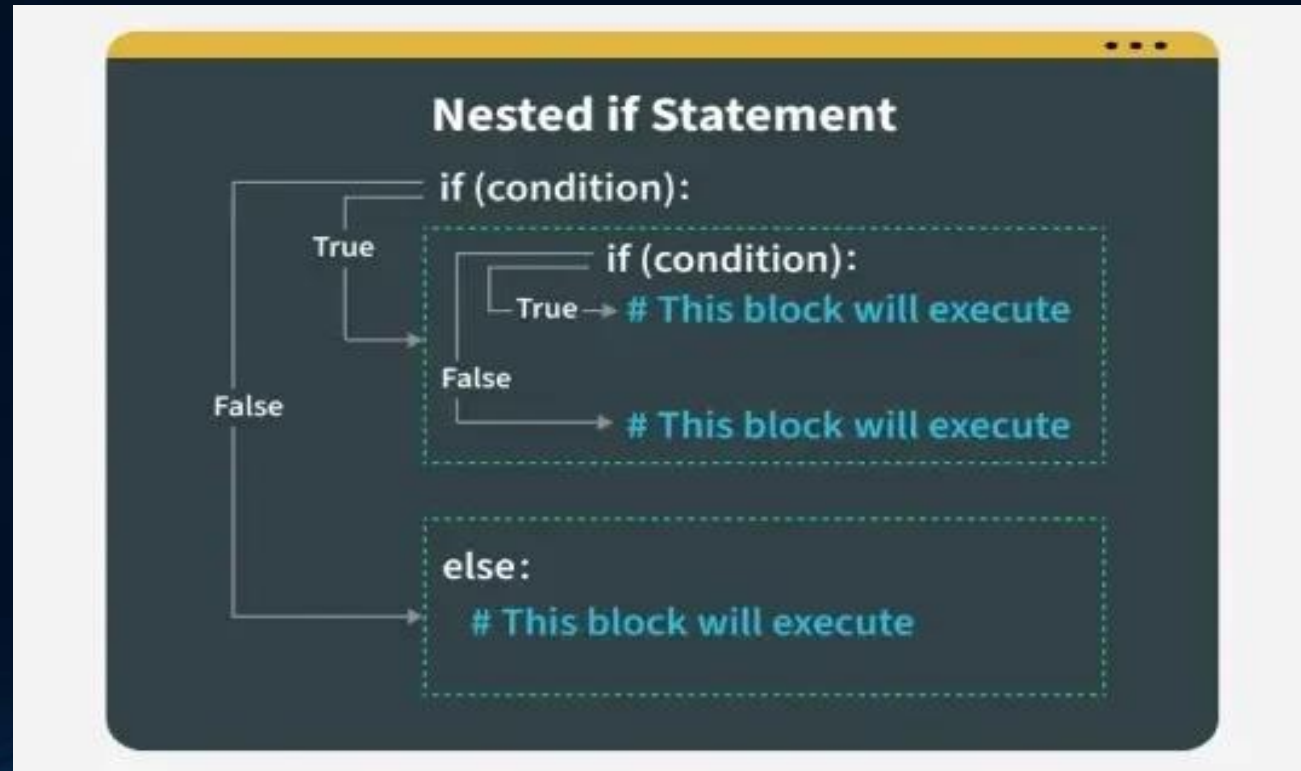
```
    print("Valid score")
```

```
else:
```

```
    print("Invalid score")
```

Nested if else Conditional Statement

- Nested if else means an if-else statement inside another if statement. We can use nested if statements to check conditions within conditions.



Example

```
age = 70
```

```
is_member = True
```

```
if age >= 60:
```

```
    if is_member:
```

```
        print("30% senior discount!")
```

```
    else:
```

```
        print("20% senior discount.")
```

```
else:
```

```
    print("Not eligible for a senior discount.")
```

Example

Three levels of nesting:

```
score = 85
attendance = 90
submitted = True

if score >= 60:
    if attendance >= 80:
        if submitted:
            print("Pass with good standing")
        else:
            print("Pass but missing assignment")
    else:
        print("Pass but low attendance")
else:
    print("Fail")
```

Ternary Conditional Statement

- A ternary conditional statement is a compact way to write an if-else condition in a single line. It's sometimes called a "conditional expression."

Assign a value based on a condition

```
age = 20
```

```
s = "Adult" if age >= 18 else "Minor"
```

```
print(s)
```

Match-Case Statement

- Match-case statement is Python's version of a switch-case found in other languages. It allows us to match a variable's value against a set of patterns.

```
number = 2
```

```
match number:
```

```
    case 1:
```

```
        print("One")
```

```
    case 2 | 3:
```

```
        print("Two or Three")
```

```
    case _:
```

```
        print("Other number")
```


Combine Values

- Use the pipe character | as an or operator in the case evaluation to check for more than one value match in one case:

Example

- `day = 4`
- `match day:`
- `case 1 | 2 | 3 | 4 | 5:`
- `print("Today is a weekday")`
- `case 6 | 7:`
- `print("I love weekends!")`

If Statements as Guards

You can add if statements in the case evaluation as an extra condition-check:

Example

```
month = 5
day = 4
match day:
    case 1 | 2 | 3 | 4 | 5 if month == 4:
        print("A weekday in April")
    case 1 | 2 | 3 | 4 | 5 if month == 5:
        print("A weekday in May")
    case _:
        print("No match")
```

Final Lab Task

- Ask the user to enter a transaction ID number. Print whether it is **Even** or **Odd**.
- If account balance is less than 500, show "Low Balance Warning", otherwise show "Balance is sufficient".
- Transaction amount > 100,000 AND Location is not "Pakistan" → Show "Fraud Alert!"
- If payment method = "card" and amount > 5000 → 10% discount If method = "wallet" → 5% discount Otherwise → No discount
- Write a program to check if a year entered by the user is a leap year.